Relatório Técnico – Desafio 4.0 (Projeto e Análise de Algoritmos)

Equipe LogisticaMA

2 de março de 2026

Resumo

Este documento descreve a solução desenvolvida para responder consultas do tipo *"quantos pacotes atrasaram mais de 30 minutos em uma janela de tempo"* com tempo de resposta inferior a 3 segundos, mesmo com até dois milhões de eventos. Propõe-se um índice baseado em ordenação temporal e prefix sums por hub e status, reduzindo o custo de consulta de $\Theta(n)$ para $\Theta(\log n)$. São apresentados a modelagem do problema, análise de complexidade temporal e espacial, metodologia experimental, resultados de benchmarks e conclusão sobre a conformidade com o SLA do desafio.

1 Introdução

O desafio consiste em acelerar consultas sobre logs logísticos que, na abordagem ingênua, percorrem toda a base. Em volumetrias acima de 10^6 registros, o tempo de resposta torna-se incompatível com uso operacional. O objetivo é garantir latência < 3 segundos para janelas arbitrárias, mantendo corretude dos resultados quando comparados ao baseline linear.

2 Requisitos do Desafio

- Responder consultas de atraso > 30 minutos em janelas de tempo arbitrárias com SLA de 3s até 2M de eventos.
- Permitir filtros por hub e por status (e combinações).
- Manter equivalência de resultados com a contagem linear de referência.
- Disponibilizar benchmark comparativo e dashboard exploratório.

3 Dataset e Normalização

A base oficial localiza-se em `data/logistica_ma_logs.csv`, com colunas obrigatórias {`id`, `timestamp`, `status`, `hub`, `atraso_min`}. A normalização converte timestamps para `timestamp_epoch` (int64) e cria rótulos legíveis. Ingestões CSV/JSON compartilham o mesmo contrato.

4 Solução Proposta

4.1 Pré-processamento

1. Ordenação estável por `timestamp_epoch` (custo $\Theta(n \log n)$).
2. Construção de prefix sums: `delayed_30_prefix` para atrasos > 30, soma acumulada e máximo acumulado de `atraso_min` (custo $\Theta(n)$).
3. Índices por hub, por status e por par (hub, status), cada um com os vetores acima.

4.2 Consulta

Dada uma janela `[start, end]`, aplica-se busca binária para localizar os limites em cada índice ($\Theta(\log n)$) e obtém-se a resposta por diferença de prefix sums ($\Theta(1)$). Para `min_delay` diferente de 30, aplica-se varredura vetorial local em `O(k)` onde k é o tamanho da janela filtrada (restrito ao subconjunto temporal), ainda dominado pela busca.

5 Análise de Complexidade

5.1 Tempo

- Pré-processamento: $\Theta(n \log n)$ (ordenar) + $\Theta(n)$ (prefixos).
- Consulta: $\Theta(\log n)$ para localizar limites + $\Theta(1)$ para diferença; $\Theta(k)$ adicional apenas se `min_delay` $\neq 30$ (janela restrita).

5.2 Espaço

- Vetores adicionais por índice: $\Theta(n)$ no pior caso, proporcional ao número de linhas. Em bytes, 3 vetores int64 por linha (timestamps e prefixos) e um int32 de atraso por linha.

6 Validação e Testes

- **Corretude de contagem**: comparação índice vs. baseline linear no dataset oficial e em amostras sintéticas.
- **Filtros**: testes unitários para filtros por hub e por status, além de `min_delay` variável.
- **Dataset**: ingestão CSV/JSON com contrato comum; dataset oficial carregado sem upload manual.
- **Benchmark CLI**: verificação automática de SLA para $n = 2{,}000{,}000$ e checagem de mesma resposta que o baseline.

7 Metodologia Experimental

Os benchmarks utilizam dados sintéticos baseados no perfil real (categorias de hub e status e distribuição de atrasos). Para cada tamanho em {1k, 10k, 100k, 1M, 2M}:

1. Gerar frame sintético com semente fixa.
2. Construir índice e medir `build_seconds`.
3. Medir consulta linear e indexada na mesma janela temporal.
4. Verificar igualdade de respostas e calcular speedup.
5. Aplicar asserção de SLA (<3s) no maior caso.

8 Resultados

Resultados da execução de `uv run python benchmarks/run_benchmarks.py` (02/03/2026):

n	build (s)	linear (s)	indexado (s)	speedup (×)	mesma resposta
1,000	0.003221	0.000043	0.000014	2.98	sim
10,000	0.005407	0.000408	0.000013	31.55	sim
100,000	0.047054	0.004412	0.000019	232.73	sim
1,000,000	0.597807	0.042632	0.000029	1461.65	sim
2,000,000	1.179973	0.085782	0.000055	1571.59	sim

Tabela 1: Benchmark sintético com perfil real. O caso de 2M satisfaz SLA de 3s.

9 Conclusões

A estrutura indexada reduz o custo de consulta de $\Theta(n)$ para $\Theta(\log n)$, permitindo respostas sub-milisegundo mesmo com 2M eventos sintéticos, atendendo o SLA de 3 segundos com ampla folga. Testes automatizados confirmam equivalência com o baseline linear e cobertura de filtros por hub, status e limiares de atraso variáveis.

10 Trabalho Futuro

- Persistir índices em disco para reuso entre execuções.
- Adicionar estimativas de uso de memória por tamanho de dataset.
- Explorar paralelização na construção dos índices por hub/status.

11 Como Executar

1. Instalar dependências: `uv sync`
2. Testes: `uv run pytest`
3. Benchmarks (com SLA): `uv run python benchmarks/run_benchmarks.py`
4. Dashboard: `uv run streamlit run app/streamlit_app.py`